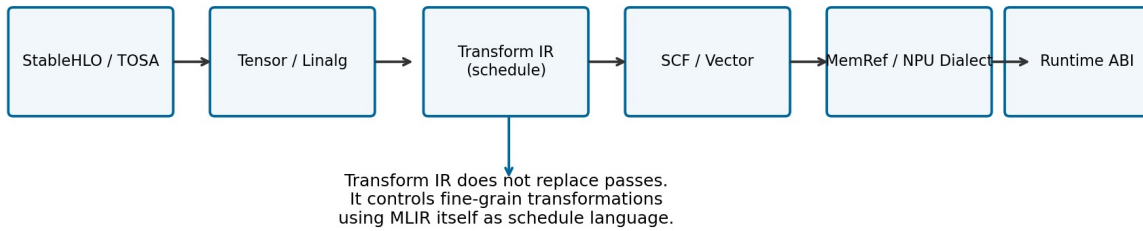


AI / NPU Compiler 를 위한 MLIR 강의

Lecture 15. Transform Dialect

tile / fuse / vectorize schedule IR for NPU compiler

Transform Dialect Position in NPU Compiler



1. 강의 목표와 위치

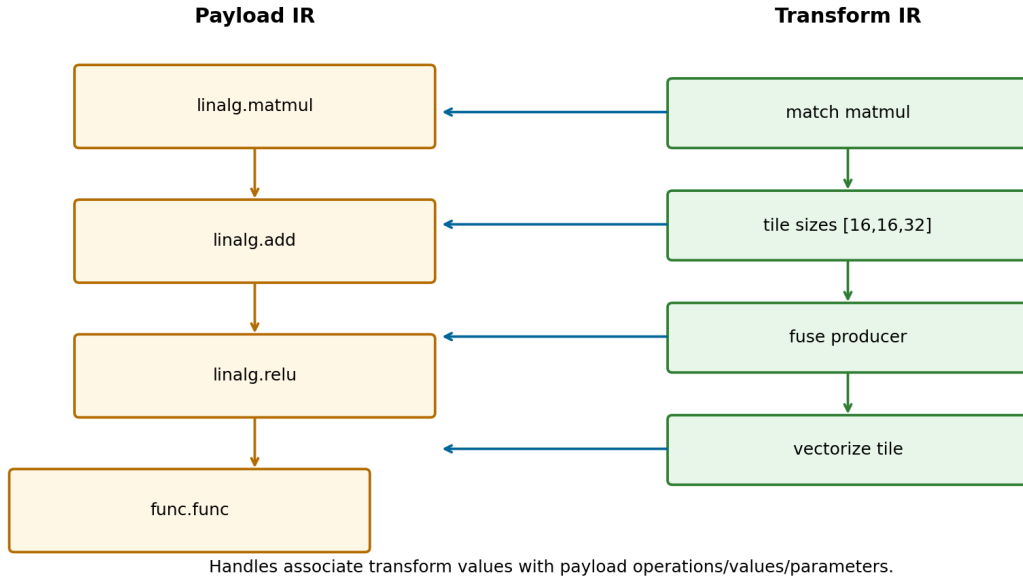
- Transform Dialect 의 payload IR, transform IR, handle, parameter 개념을 설명한다.
- Linalg MatMul/Conv 에 대해 tile, fuse, vectorize schedule 을 IR 로 표현한다.
- schedule IR 을 NPU tile shape, SRAM footprint, accumulator type, DMA policy 와 연결한다.
- transform-interpreter 기반 실행 흐름과 regression test 전략을 설계한다.
- Transform Dialect 를 pass infrastructure 와 rewrite pattern 의 대체물이 아닌 orchestration layer 로 이해한다.

이전/현재/다음	강의	핵심 연결
14 강	Vector Dialect, Tensorization	vector.contract 를 NPU matmul/tensor instruction 으로 매핑
15 강	Transform Dialect	tile/fuse/vectorize schedule 을 IR 로 명시하고 검증
16 강	Quantization	INT8/INT4 requant, accumulator width, scale/zero-point policy

2. 핵심 개념: Transform IR 과 Payload IR

Transform Dialect 는 변환 대상 IR(payload IR)과 변환을 지시하는 IR(transform IR)을 분리합니다. NPU compiler 관점에서는 transform IR 이 schedule language 역할을 하며, tile size, fusion boundary, vector shape, fallback policy 를 명시적으로 남기는 방법입니다.

Payload IR vs Transform IR

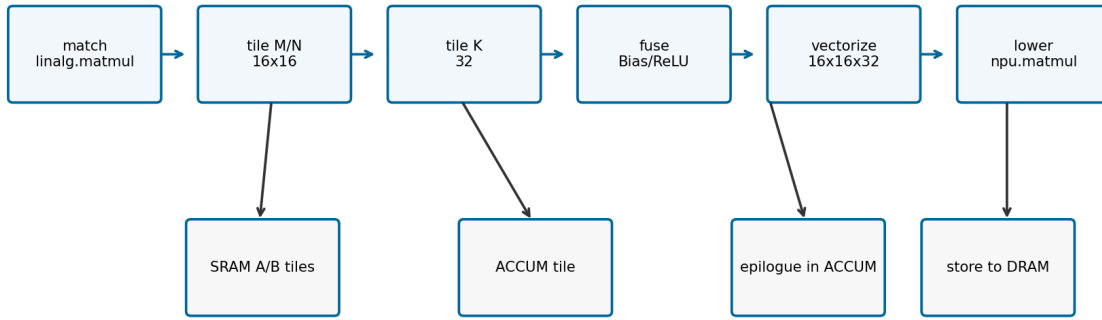


용어	의미	NPU compiler 해석
Payload IR	실제 모델/연산 IR	StableHLO/TOSA/Linalg/SCF/Vector 등 실제 lowering 대상
Transform IR	payload 를 변환하는 별도 IR	schedule IR: tile, fuse, vectorize, pack, lower policy
Handle	payload operation/value set 과 연결된 transform SSA value	MatMul op set, generated loop, vector op 등을 추적
Parameter	실행 시점에 알려진 attribute-like 값	tile size, vector size, fallback threshold

3. 왜 NPU 에서 특히 중요한가?

- NPU schedule 은 단순 최적화가 아니라 hardware contract 입니다. tile shape 는 compute array, SRAM capacity, accumulator width, DMA alignment 와 동시에 맞아야 합니다.
- 동일한 linalg.matmul 이라도 dtype, layout, batch shape, K remainder 에 따라 schedule 이 달라집니다. Transform Dialect 는 이를 pass 내부 heuristic 으로 숨기지 않고 IR 로 표현할 수 있게 합니다.
- Pass pipeline 은 큰 stage 를 담당하고, rewrite pattern 은 local rewrite 를 담당합니다. Transform IR 은 특정 payload op 를 골라 fine-grained transformation 을 순서대로 적용하는 orchestration layer 입니다.

NPU Schedule IR for MatMul + Bias + ReLU

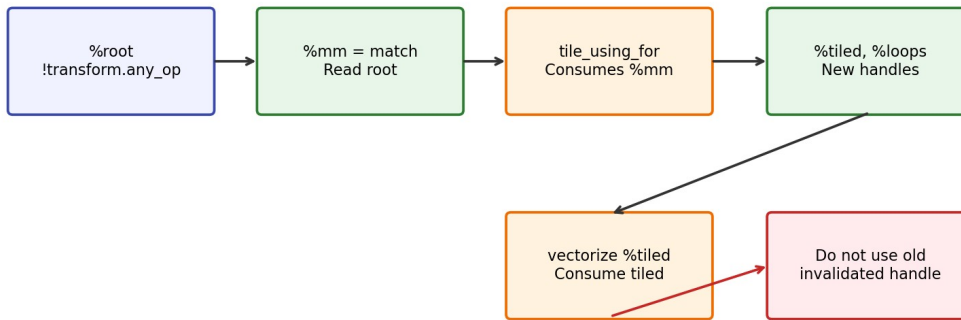


Schedule = hardware-facing policy: tile sizes, fusion boundary, vector shape, memory movement.

4. Handle Lifetime 과 Failure Mode

Transform operation 이 payload IR 을 변경하면 입력 handle 을 consume 하고 새 handle 을 반환하는 경우가 많습니다. 따라서 tile 후에는 원래 MatMul handle 을 계속 쓰지 말고, 반환된 tiled op handle 이나 loop handle 을 사용해야 합니다.

Handle Lifecycle: Read -> Consume -> Produce



NPU compiler rule: every schedule step must record which handles remain valid.

구분	의미	NPU 예시
silenceable failure	전제조건 실패. fallback 가능	K 가 native tile 32 와 맞지 않아 split/mask 필요
definite failure	semantic/postcondition 위반	IR 을 이미 변경한 뒤 invalid state 가 됨
handle invalidation	consume 된 handle 재사용 금지	tile_using_for 후 old matmul handle 로 vectorize 시도

5. 기본 Transform Schedule 예제

예제: MatMul + Epilogue schedule sketch

```
// Transform IR schedule sketch.
// Syntax is close to upstream MLIR but may need adjustment for a specific LLVM/MLIR revision.
// Check upstream Transform Dialect docs and examples/transform tests for exact current spelling.

module attributes {transform.with_named_sequence} {
  transform.named_sequence @_transform_main(%root: !transform.any_op)
    attributes {transform.readonly} {
      // 1. Select the MatMul payload operation.
      %matmul = transform.structured.match ops{["linalg.matmul"]} in %root
      : (!transform.any_op) -> !transform.any_op

      // 2. Tile M/N/K to the NPU native tile shape.
      // M=16, N=16, K=32 for INT8xINT8->INT32 accumulator.
      %tiled_matmul, %loops:3 = transform.structured.tile_using_for %matmul
      tile_sizes [16, 16, 32]
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op, !transform.any_op)

      // 3. Match epilogue op and fuse it into the containing loop/tile when legal.
      %epilogue = transform.structured.match ops{["npu_graph.bias_relu"]} in %root
      : (!transform.any_op) -> !transform.any_op
      // Pseudo: exact fusion target depends on the generated loop/tile handles.
      // %fused, %new_loop = transform.structured.fuse_into_containing_op %epilogue into %loops#0
      // : (!transform.any_op, !transform.any_op) -> (!transform.any_op, !transform.any_op)

      // 4. Vectorize the tiled MatMul. It can later be tensorized to npu_kernel.matmul.
      transform.structured.vectorize %tiled_matmul vector_sizes [16, 16, 32]
      : !transform.any_op

    }
  transform.yield
}
```

6. Tile / Fuse / Vectorize 를 NPU 의사결정으로 읽기

Transform step	Compiler effect	NPU decision
match	payload op set 선택	linalg.matmul, conv, elementwise epilogue 후보 선정
tile_using_for	loop nest + tiled op 생성	M/N/K tile, outer/inner tiling, DMA granularity 결정
fuse_into_containing_op	producer/consumer 를 tile 내부로 이동	Bias/ReLU/Requant 를 accumulator locality 안에서 처리
vectorize	Linalg op 를 vector dialect 로 변환	vector.contract -> npu_kernel.matmul tensorization 준비
apply_patterns	cleanup/canonicalization	unexpected copy, transpose, dead op 제거

7. Two-level Tiling Policy

Edge NPU 에서는 outer tile 과 inner tile 을 분리하는 것이 일반적입니다. outer tile 은 SRAM residency 와 core/cluster 분배를 결정하고, inner tile 은 native tensor instruction shape 와 매핑됩니다.

예제: two-level tiling pseudo schedule

```
// Two-level tiling pseudo schedule for edge NPU.
// L1: outer tiles assigned to cores/clusters.
// L0: inner tiles mapped to native tensor/matrix instruction.

transform.named_sequence @npu_matmul_two_level(%root: !transform.any_op)
  attributes {transform.readonly} {
    %matmul = transform.structured.match ops{["linalg.matmul"]} in %root
    : (!transform.any_op) -> !transform.any_op

    // Outer tiling for SRAM residency and parallelism.
    %outer_tiled, %outer_loops:3 = transform.structured.tile_using_for %matmul
    tile_sizes [64, 64, 64]
    : (!transform.any_op) -> (!transform.any_op, !transform.any_op, !transform.any_op, !transform.any_op)

    // Inner tiling for native NPU array.
    %inner_tiled, %inner_loops:3 = transform.structured.tile_using_for %outer_tiled
    tile_sizes [16, 16, 32]
    : (!transform.any_op) -> (!transform.any_op, !transform.any_op, !transform.any_op, !transform.any_op)

    transform.structured.vectorize %inner_tiled vector_sizes [16, 16, 32]
    : !transform.any_op
    transform.yield
  }
```

항목	Outer tile 예시	Inner tile 예시	의미
M/N	64 x 64	16 x 16	SRAM chunk vs compute array
K	64	32	reduction chunk vs native K width
parallelism	core/cluster 분배	instruction-level tile	multi-core 와 tensor op 분리
edge tile	split/peel	mask/unroll	partial tile 정책 분리

8. SRAM / ACCUM Footprint 계산

Buffer	Formula	INT8/INT32 예시 M=16,N=16,K=32
A tile	$M_tile * K_tile * \text{bytes}(A)$	$16 * 32 * 1 = 512 \text{ B}$
B tile	$K_tile * N_tile * \text{bytes}(B)$	$32 * 16 * 1 = 512 \text{ B}$
ACC tile	$M_tile * N_tile * \text{bytes}(\text{acc})$	$16 * 16 * 4 = 1024 \text{ B}$
Double buffer	$2 * (A \text{ tile} + B \text{ tile}) + \text{ACC}$	$2 * (512 + 512) + 1024 = 3072 \text{ B}$

9. Schedule Parameter YAML

예제: NPU schedule spec

```
# NPU schedule spec generated from Transform Dialect design exercise.
schedule_name: matmul_bias_relu_int8
payload_match:
  op: linalg.matmul
  lhs_type: tensor<MxKxi8>
  rhs_type: tensor<KxNxi8>
  acc_type: tensor<MxNxi32>
policy:
  outer_tile: {M: 64, N: 64, K: 64}
  inner_tile: {M: 16, N: 16, K: 32}
  vector_contract: {lhs: "16x32xi8", rhs: "32x16xi8", acc: "16x16xi32"}
  epilogue_fusion: [bias, requant, relu]
  memory:
    A_tile_space: SRAM0
    B_tile_space: SRAM1
    C_acc_space: ACCUM
    double_buffer: true
  fallback:
    unsupported_layout: repack_or_cpu
    k_remainder: split_or_mask
    dynamic_shape: runtime_tile_selection
checks:
  max_sram_bytes_per_core: 262144
  max_accumulator_bytes: 65536
  require_dma_alignment_bytes: 64
```

10. Regression Test Strategy

Transform Dialect Regression Matrix

Schedule	Expected IR	Legality	Perf Proxy
tile sizes tile sizes	CHECK	CHECK	CHECK
fusion fusion	CHECK	CHECK	CHECK
vectorization vectorization	CHECK	CHECK	CHECK
failure path failure path	CHECK	CHECK	CHECK

For NPU: schedule correctness is not enough; check memory footprint, vector shape, and fallback cases.

Test type	검증 내용	CHECK 예시
positive schedule	expected scf/vector/npv op 존재	CHECK: scf.for, CHECK: vector.contract
negative legality	unsupported shape/layout diagnostic	expected-error 또는 CHECK: fallback
copy regression	unexpected memref.copy 방지	CHECK-NOT: memref.copy
handle safety	use-after-free 가능성 탐지	-transform-dialect-check-uses

예제: FileCheck template

```
// RUN: mlir-opt %s --transform-interpretter --canonicalize | FileCheck %s
// NOTE: This is a template. Adjust exact CHECK lines after choosing an MLIR version.

// CHECK-LABEL: func.func @matmul_bias_relu
// CHECK: scf.for
```

```
// CHECK: vector.contract
// CHECK-NOT: memref.copy

module attributes {transform.with_named_sequence} {
  // Payload and transform schedule would be placed here in a real test.
}
```

11. Custom NPU Transform Operation 설계

기존 structured transform op 로 표현하기 어려운 target-specific schedule 은 downstream TransformDialectExtension 으로 확장할 수 있습니다. 예를 들어 npu.schedule.matmul_tile, npu.schedule.promote_to_sram, npu.schedule.tensorize_to_npu 같은 operation 을 정의하면 schedule IR 과 NPU legality checker 를 직접 연결할 수 있습니다.

예제: out-of-tree NPU transform extension skeleton

```
// Pseudo C++ skeleton for an out-of-tree NPU Transform Dialect extension.
// This is a teaching artifact, not a drop-in build file.

#include "mlir/Dialect/Transform/IR/TransformDialect.h"
#include "mlir/Dialect/Transform/Interfaces/TransformInterfaces.h"

namespace npu {
class NPUTransformDialectExtension
: public mlir::transform::TransformDialectExtension<NPUTransformDialectExtension> {
public:
  NPUTransformDialectExtension() {
    // declareGeneratedDialect<npu::NPUKernelDialect>();
    // registerTransformOps<ScheduleMatmulTileOp, PromoteToSramOp, TensorizeToNpuOp>();
  }
};
}

// Design sketch:
// npu.schedule.matmul_tile %target {m=16, n=16, k=32}
// npu.schedule.promote_to_sram %tile {space="SRAM0", double_buffer=true}
// npu.schedule.tensorize_to_npu %vector_contract {instruction="mma_i8_i32"}
```

12. 실습

1. 실습 A: payload IR 에서 MatMul/Add/ReLU op 를 찾고 각 handle 의 payload association 을 표로 작성합니다.
2. 실습 B: tile_using_for, fuse_into_containing_op, vectorize 순서로 schedule IR 을 작성합니다.
3. 실습 C: native tile 16x16x32 기준으로 SRAM footprint 와 fallback policy 를 작성합니다.
4. 실습 D: FileCheck positive/negative test 를 작성합니다.

13. Common Mistakes

- tile 후 반환된 tiled handle 이 아니라 기존 matmul handle 을 후속 transform 에 사용하는 실수.
- vectorize 를 너무 일찍 적용하여 fusion 이나 tiling 기회를 잃는 실수.
- tile size 만 맞추고 SRAM/ACCUM footprint 또는 DMA alignment 를 검증하지 않는 실수.
- Transform Dialect 를 pass infrastructure 의 대체물로 오해하는 실수. 실제로는 pass/pattern 을 호출하고 조합하는 orchestration layer 에 가깝습니다.
- 실패 경로를 설계하지 않아 unsupported shape 에서 조용히 잘못된 IR 을 만드는 실수.

14. 과제

과제	산출물	평가 기준
MatMul+Bias+ReLU schedule 작성	transform IR + YAML schedule spec	tile/fuse/vectorize 순서와 handle lifetime 정확성
Conv2D schedule variant	NHWC/HWCF conv transform sketch	spatial/channel/reduction tile 정책
Negative test	unsupported K remainder FileCheck	diagnostic/fallback policy 명확성
NPU custom op 제안	npu.schedule.* op schema	operand/result/attribute/legality 정의

15. References

MLIR Transform Dialect: <https://mlir.llvm.org/docs/Dialects/Transform/>

MLIR Transform Dialect Tutorial: <https://mlir.llvm.org/docs/Tutorials/transform/>

Transform Tutorial Chapter 1: <https://mlir.llvm.org/docs/Tutorials/transform/Ch1/>

Transform Tutorial Chapter H - Halide Schedule: <https://mlir.llvm.org/docs/Tutorials/transform/ChH/>

MLIR Linalg Dialect: <https://mlir.llvm.org/docs/Dialects/Linalg/>

MLIR Bufferization: <https://mlir.llvm.org/docs/Bufferization/>

MLIR Passes - Transform Interpreter: <https://mlir.llvm.org/docs/Passes/>